Autonomous Quadruped Multi-Agent SLAM and Exploration

Design, implementation and baseline evaluation

Project report | course project 23CSE399, Amrita Vishwa Vidyapeetham | prepared 25 October 2026 | baseline run 20261024_210638 | code: snapshot commit 6f9c01e; the run recorded 1f231d2 plus uncommitted changes

Abstract

This report describes a CPU-only system that lets a legged robot explore an unknown multi-room indoor space on its own: it estimates where it is, builds a map, finds rooms, doorways and objects, decides where to go next, plans a path and walks it. The system is split into seven cooperating software agents and runs in the CoppeliaSim simulator. It was built as the navigation half of a first-response robot for crime-scene exploration. The report also describes the measurement framework that was built to evaluate the system honestly, and presents a complete baseline: before any improvement, the system was run, scored against simulator ground truth and against the real geometry of the scene, and compared with published systems and commercial platforms.

In one run of 1800 simulated seconds (157.9 m travelled) the localization error was 31.0 cm RMSE (0.20% of the path), the map had 10.6 cm accuracy against the real walls, and 88% of the real floor was mapped. The robot used 86% of one CPU core and a 0.19 MB map. However, the mission did **not complete**: 6 of 8 rooms were finished when the time limit was reached, the robot walks slowly (0.088 m/s), and its object detection is a size rule rather than recognition. The baseline identifies where the system is strong (drift, cost, component contributions), where it is behind published systems (absolute accuracy, speed, completion), and exactly what to fix next.

Contents

1 Introduction | 2 System overview | 3 Methods: how each agent works | 4 Evaluation methodology | 5 Experimental setup | 6 Results | 7 Comparison with published systems and the market | 8 Analysis and discussion | 9 Roadmap | 10 Conclusion | References | Appendices A to C

1. Introduction

1.1 The problem

When responders must enter an unknown or dangerous indoor space (a hostage situation, a crime scene, a collapsed building), a person entering first is at risk and may disturb evidence. A small legged robot can go in first, but it has to cope without GPS, without a prior map and without an operator steering it: it must work out where it is while it builds the map (SLAM) [14], decide what to look at next, and avoid walls and obstacles. Doing this on a small computer, without a GPU, is part of the requirement.

1.2 What this project is

The course project 23CSE399 defines a seven-stage first-response pipeline: sense, navigate, trigger, perceive, decide, store and sync. This repository implements the first two stages, **Sense and Navigate**: a quadruped that explores an unknown multi-room space with SLAM and frontier-based exploration, in simulation. The later stages (an anomaly-detection model, event triggering, forensic logging with hashing and encryption, opportunistic synchronization, battery-aware return) are not part of this code base and are listed in the roadmap.

1.3 Objectives of this work

- Build the sense-and-navigate core as a set of cooperating agents (Section 2 and 3).
- Measure its performance honestly, from real runs, with no assumed or typed-in numbers: overall, per agent, localization and mapping separately, and compute cost (Section 4).
- Establish a baseline before changing anything, so every later improvement can be judged against it (Sections 5 and 6).
- Compare the result fairly with published SLAM and exploration systems and with commercial legged robots, and say what the comparison does and does not show (Section 7).
- Record every weakness found and the planned fix (Sections 8 and 9).

1.4 Contributions

- A seven-agent, CPU-only exploration and SLAM stack for a simulated quadruped, with no use of simulator ground truth by the estimator (verified automatically).
- A measurement framework: per-run telemetry, ground-truth scoring against the real scene geometry, offline replay and ablation, an integrity audit, and automatic reports.
- A documented baseline with 38 classified metrics, a component ablation, a comparison with published systems on common metrics and a capability comparison with six commercial platforms.
- A prioritized register of 31 findings and fixes with the evidence behind each.

2. System overview

2.1 What the robot does

The robot starts at a known position in an unknown building. It looks forward with a depth camera, and repeatedly: (1) converts the depth image to a 2D picture of the obstacles around it, (2) works out how far it moved by matching consecutive pictures and combining that with its leg-gait odometry and a gyro, (3) adds what it sees to a grid map, (4) finds the boundary between explored and unexplored space, (5) chooses the most useful boundary to visit next, (6) plans a collision-free path, and (7) walks it. It splits the map into rooms and doorways, sweeps one room at a time, moves to the next room through a doorway, and stops when it believes no reachable unexplored space is left.

2.2 Inputs

Table 1. Inputs to the system.

Input	Source	Used for
Depth image and colour image	Simulated Kinect-style RGB-D camera facing forward (60 degree field of view, 0.1 to 8 m range)	Obstacle scan and map; the colour image is recorded but not used for detection
Gait odometry	Commanded gait speed converted to body speed through a calibrated curve	Predicted forward motion between scans
Gyro	Simulated rate gyro (0.10 deg/s noise, 0.05 deg/s bias, fixed seed)	Heading change between scans
Start pose	Initial robot position and heading	Origin of the pose estimate
Mission limits	Command-line options (time limit, headless)	Safety stop of a run

The estimator never receives the simulator's robot pose. Ground truth is read separately, only for scoring (Section 4).

2.3 Outputs

Table 2. Outputs of the system.

Output	Form	Produced by
Robot pose	x, y, heading at 10 Hz (topics pose/slam, pose/state)	SLAM agent
Occupancy map	220 x 220 grid, 5 cm cells, log-odds	Mapping agent
Rooms and doorways	Room labels on the grid; doorway positions	Mapping agent
Exploration target and path	Target cell; list of waypoints	Frontier and Planning agents
Motion commands	Gait speed and steering each step	Locomotion agent
Knowledge graph and event log	Rooms, doorways, objects with positions; timeline of events	Supervisor agent
Run record	Metrics, per-step trajectory, raw sensor recording, manifest, report	Measurement framework

2.4 Architecture

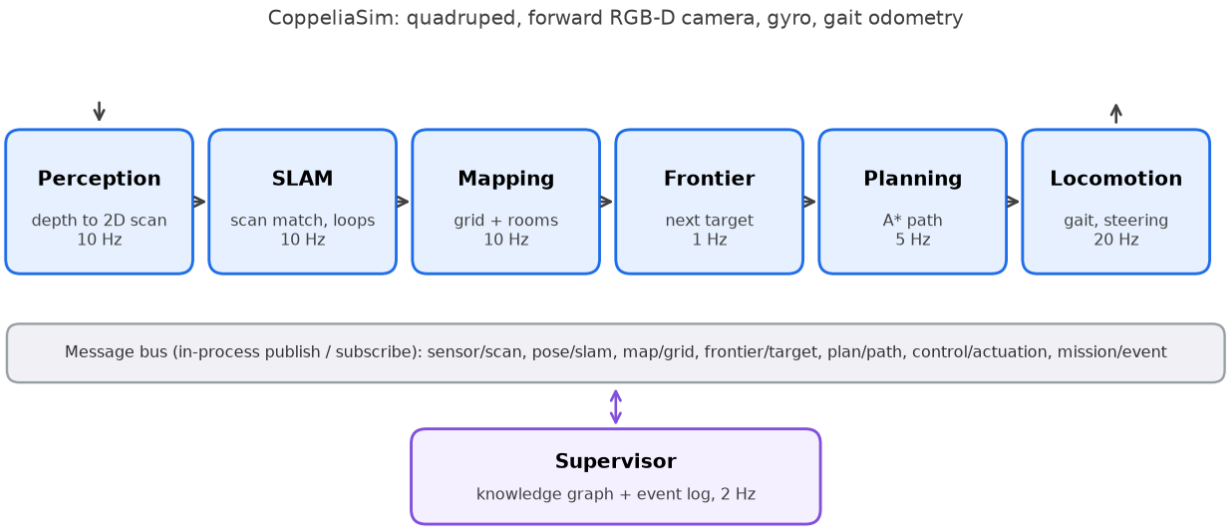


Figure 1. Agent architecture. Agents run in one process and exchange messages over an in-process publish/subscribe bus.

2.5 The seven agents

Table 3. Agents, rates and responsibilities.

Agent	Rate	Responsibility	Main output
Perception	10 Hz	Converts each depth image into 3D points in the robot's body frame, removes points on the robot itself and keeps a height band of 0.04 to 0.60 m	2D obstacle scan, 3D points
SLAM	10 Hz	Estimates the pose by scan matching seeded with odometry, fuses heading with the gyro and a wall-direction prior, closes loops on revisits	Pose estimate
Mapping	10 Hz	Integrates scans into a probabilistic grid, segments rooms, detects doorways, labels obstacle blobs	Grid, rooms, doorways, objects
Frontier	1 Hz	Finds unexplored boundaries, scores them with a geodesic cost field, sweeps room by room, decides when exploration is complete	Next target
Planning	5 Hz	Plans a collision-free A* path over known free space and smooths it	Waypoints

Agent	Rate	Responsibility	Main output
Locomotion	20 Hz	Generates the walking gait, steers to the waypoint, detects lack of progress and recovers, reports odometry and gyro	Gait commands, odometry
Supervisor	2 Hz	Maintains the knowledge graph and event timeline	Graph, timeline

2.6 Execution model (what 'agent' means here)

The agents are Python classes that share one interface (initialize, step, shutdown). One loop in the main program calls each agent's step in a fixed order at that agent's rate, then advances the simulator by one step. Messages travel over an in-process bus: a publisher stores the latest message per topic and immediately calls its subscribers. Everything runs in one process on one thread. There is no robotics framework (such as ROS 2) and no parallel execution, and the supervisor is a fixed state machine, not a learned or language-model agent. The benefit is that each concern is isolated, testable and replaceable. The cost, measured in Section 6.6, is that the agents account for only 46% of the wall-clock time; the rest is simulator stepping, image transfer and display.

Table 4. Message bus topics.

Topic	Content	Publisher
sensor/rgbd, sensor/scan	Depth and colour image; body-frame obstacle scan	Perception
pose/slam, pose/state	Estimated pose (pose/state is the pose the rest of the system navigates on)	SLAM
map/grid	Grid, free/occupied counts, room labels, doorways	Mapping
frontier/target	Selected exploration target	Frontier
plan/path	Waypoint list	Planning
control/actuation	Gait speed, heading error, odometry increment, gyro increment	Locomotion
mission/event	Room entered or completed, object detected, loop closure, stuck, target unreachable	Several agents

3. Methods: how each agent works

3.1 Perception

The depth image is back-projected into 3D points using the camera's focal length (from the 60 degree field of view) and its fixed mounting transform on the robot body, so points are expressed directly in the robot's body frame without using any world pose. Points within 22 cm of the body centre are discarded (the robot's own body and legs). For mapping, points between -0.08 m and 0.60 m height are kept; for the 2D obstacle scan, points at least 0.04 m above the floor are kept and every fourth point is used (about 600 points per scan in this run). This height band is why objects lower than 4 cm or higher than 60 cm are not seen.

3.2 Odometry and gyro

The locomotion agent converts the commanded gait speed into a body speed through a piecewise-linear curve calibrated offline: commands 0, 0.35, 0.75 and 1.0 map to 0, 0.047, 0.053 and 0.0785 m/s. The curve was calibrated once from an early recording that used ground truth; this is disclosed as a calibration, and the estimator itself does not read ground truth at run time. The gyro is simulated from the true orientation change per step plus white noise and a constant bias, so heading drifts as a real inexpensive gyro would.

3.3 SLAM

Scan matching. Each new scan is aligned to the previous one with point-to-line ICP, a variant [17] of the iterative closest point method [16] that uses wall-line normals, seeded with the odometry prediction. A match is accepted only if the residual is at most 5 cm, the step is at most 0.8 m and 120 degrees, and the result agrees with odometry to within 1 cm plus the step length. While walking, rotation is held at the gyro value and only translation comes from ICP, because with a narrow field of view rotation and sideways slide are nearly indistinguishable. During slow turns ICP may correct heading with a weight of 0.1 when it agrees with the gyro to within 3 degrees. When a match is rejected, the pose advances by odometry alone.

Degeneracy filter. Along a featureless wall the match cannot determine motion parallel to it; the filter removes the ICP correction along that direction.

Manhattan heading prior. Indoor walls meet at right angles (the Manhattan-world assumption [20]). After each accepted match the dominant wall direction is found from the scan normals and the heading is nudged toward the nearest multiple of 90 degrees (gain 0.1, only when within 15 degrees). This cancels gyro drift.

Loop closure. A keyframe (scan plus pose) is stored every 0.5 m or 25 degrees. When the robot returns within 2.5 m of an older keyframe (at least 12 s older), the current scan is matched to it; the closure is accepted only for a residual of at most 3 cm and an overlap of at least 60%, and the correction (limited to 0.3 m) is spread over the pose graph [18], in the manner of loop closure in 2D LiDAR SLAM [19].

3.4 Mapping

Each scan is ray-cast from the estimated pose into a log-odds occupancy grid [21, 22] of 220 x 220 cells of 5 cm (an 11 m x 11 m area). A cell on a ray's end gains 0.85, cells along the ray lose 0.4, and values are limited to plus or minus 2. A cell is occupied above 1.0 (about two hits) and free below -0.5. Rooms are found by thresholding the distance transform of the free space into room cores and partitioning the grid by distance (a Voronoi partition); a doorway is where two different rooms touch in free space. Connected obstacle blobs are labelled 'wall' if at least 1.2 m across and 'chair' if 0.18 to 1.05 m across. This labelling is purely geometric.

3.5 Frontier selection

A frontier [23] is a free cell next to unknown space; frontiers are grouped into clusters. For each cluster the agent computes the walking distance over inflated free space with a breadth-first wavefront (clusters behind walls get infinite distance and are skipped), then scores it as information gain (cluster size plus 0.3 per unknown cell in a 9 x 9 window) minus 3.8 times the walking distance, with a bonus for clearance and a penalty for turning around, so the robot sweeps in one rotational direction. The agent works room by room: while the current room has reachable frontiers it sweeps them, then it marks the room complete and moves to the nearest unvisited doorway. Targets are kept for up to 9 s; unreachable or stalled targets are blacklisted. Exploration is declared complete when no reachable frontier remains.

3.6 Planning

A* [24] (8-connected) runs on the known free space with a hard 15 cm clearance around obstacles (obstacles dilated by three cells) and a soft penalty that prefers staying away from walls. Paths are smoothed by replacing waypoints with straight lines whenever the line is free of inflated obstacles. Planning only uses known free cells, so the robot never plans through unknown space.

3.7 Locomotion

The gait is a sinusoidal pattern generator driving the leg joints, with frequency and amplitude scaled by the commanded speed. The speed command depends on heading error: 1.0 for straight walking, 0.75 for walking while turning and 0.35 for turning in place. Two safeguards run continuously: every second the agent checks that the robot has moved at least 5 cm while walking, and after two failed checks it executes a short reverse step; and if the robot has not come closer to its goal for 15 s the goal is dropped. Turning is done by setting the body orientation toward the desired heading (a simulation shortcut, recorded as a limitation).

3.8 Supervisor

The supervisor listens to mission events and the map, and maintains a knowledge graph with three kinds of nodes (rooms, doorways, objects) linked by 'located in' edges, together with a time-ordered event log. It exports both at the end of a run.

4. Evaluation methodology

4.1 Principles

- Every reported value is computed from the run's own recorded data; none is typed in or taken from an earlier experiment. A value that cannot be measured is reported as unavailable with the reason.
- A measured result, an estimate and a diagnostic indicator are labelled differently; single-run results carry no claim of repeatability.
- Localization and mapping are evaluated separately, then together.
- Component contributions are measured only by offline replay of recorded data, never by switching components off in the live robot.
- The measurement must not disturb the robot: it only wraps agent calls with a timer, writes files after the control loop, and bounds its memory.

4.2 Ground truth and references

Table 5. What each result is scored against.

Quantity	Reference	Notes
Position and heading	Simulator pose of the robot body at every control step (0.05 s)	Used for scoring only. No trajectory alignment is applied because SLAM starts at the true start pose
Map, first reference	A second grid built from the same depth rays using the true pose	Isolates the effect of pose error; limited to what the sensor saw
Map, independent reference	Wall and object geometry read directly from the CoppeliaSim scene	Footprints are rasterized at 5 cm; independent of the robot's sensor
Objects	Positions and sizes of the 13 scene objects in the sensing height band	A detection counts if within 0.5 m of an object
Floor coverage	Real floor area of the arena (interior 9.9 x 9.9 m)	Share of real floor marked free in the map

4.3 Metrics

Table 6. Metric definitions (units in parentheses).

Metric	Definition
ATE RMSE (cm)	Root-mean-square Euclidean distance between estimated and true position over every control step [13]
ATE as share of path (%)	ATE RMSE divided by the true distance travelled; lets runs of different length be compared
Final drift (% of path)	Position error at the end of the run divided by the distance travelled
RPE (% and deg/m)	Error of the relative motion over every 1 m of true path [13, 25], expressed in the body frame at the start of the segment; translation as percentage of the segment, rotation per metre
Scan-match acceptance (%)	Share of SLAM updates in which the scan match passed its acceptance checks (a diagnostic, not a failure rate)
Map accuracy / completion (cm)	Mean distance from each mapped obstacle cell to the nearest reference obstacle cell / from each reference cell to the nearest mapped cell

Metric	Definition
Precision, recall, F1 (%)	Agreement of obstacle cells within a 15 cm tolerance: mapped cells near a reference cell / reference cells near a mapped cell / their harmonic mean
Completion ratio (%)	Share of reference obstacle cells matched within 5 cm
Free-space IoU (%)	Intersection over union of free cells between map and reference
Detection precision (%)	Share of reported object detections within 0.5 m of a real object
Latency (ms)	Wall time of one agent step call: median, 95th percentile, maximum
Real-time factor	Simulated seconds per wall-clock second; 1 means real time
Process CPU / memory	CPU and resident memory of the Python process, sampled once per second; the simulator is excluded

4.4 Measurement framework

Every run writes a self-contained folder: the per-step trajectory of truth and estimate, all metrics, the per-agent table, a manifest (code version, computer, configuration, metric definitions), a recording of exactly what the SLAM agent received each step, the grids and knowledge graph, and an automatic report. The recording allows any SLAM configuration to be replayed offline in minutes on identical data, which is how component contributions are measured. Replay validity was checked: the full-system replay matches the live trajectory to within 0.01 cm over 36001 steps, and two identical replays are bit-for-bit equal. The estimated cost of the measurement itself is 0.0034 ms per step (about 0.004% of the median control step); this is a calibrated estimate, not an uninstrumented comparison run.

4.5 Integrity audit

Four automatic checks accompany every report: no simulator robot-pose call exists in the SLAM, mapping, perception, frontier, planning or odometry source (static scan of the code); the SLAM agent does not subscribe to ground truth in closed-loop mode; the estimated trajectory is not a copy of ground truth; and the run navigated on the SLAM pose. All four pass for the baseline run.

4.6 Classification of results

To say where the system stands, each metric is rated Good, Improve or Poor. The ratings are indicative: they come from engineering targets and from published numbers and product specifications, are not published standards, and can be changed. They are kept in the internal diagnostic report and are not part of this report.

5. Experimental setup

Table 7. Setup of the baseline run.

Item	Value
Simulator and scene	CoppeliaSim EDU [15]; a 10 m x 10 m arena with four outer walls, 15 interior wall segments forming 8 rooms and corridors, and 13 objects (table, laptop, chair, cupboard, plant, crates, cylinders, two human figures, three beacons)
Robot	Quadruped with sinusoidal gait; forward RGB-D camera; simulated gyro
Computer	AMD Ryzen 7 7735HS with Radeon Graphics (16 threads); GPU present but unused; Windows-11-10.0.26200-SP0; Python 3.12.10
Run	One run, safety limit 1800 simulated seconds, ended: timeout (1800s) before all frontiers were cleared
Duration	1800 s simulated in 5666 s of wall time
Recording	All 36,001 control steps recorded (0.05 s each)
Number of runs reported	1. No variability can be stated

6. Results

6.1 Summary

Of 38 classified metrics, **10 are Good, 12 can be improved and 16 are Poor**. In short: localization drift, computational cost and the contribution of the SLAM components are good; absolute accuracy and map precision can be improved; mission completion, speed, local pose error, object detection and real-time operation are poor.

6.2 Task and exploration

Table 8. Task and exploration results.

Metric	Value	What it measures
Mission completed	no	Mission reported every reachable frontier explored
Simulated time	1800 s	Stopped at the safety limit
Distance travelled	157.9 m	True path length
Average speed	0.088 m/s	Distance divided by time
Rooms completed / found	6 / 8	Rooms marked fully explored / rooms segmented
Real floor mapped	88.2% of 85.2 m2	True coverage of the arena floor
Coverage plateau	97% of final coverage at 930 s	After this time only 2.1 points of coverage were added
Path walked after the plateau	73.2 m (46%)	Effort that added little
Frontiers remaining	48	Unexplored boundaries left at the end
Stuck events / unreachable targets	4 / 9	Watchdog and planner events
Rooms entered per room	2.25	18 entry events for 8 rooms

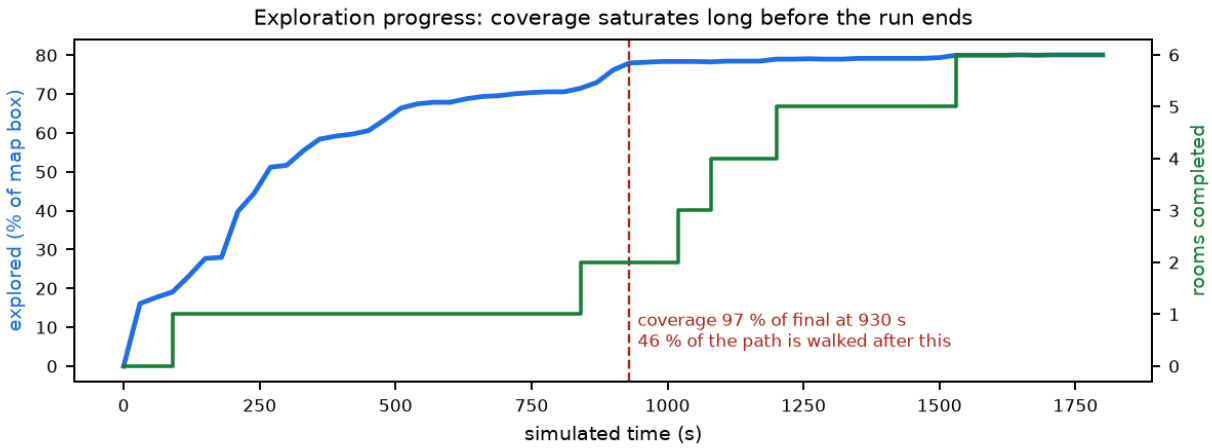


Figure 2. Exploration progress. Coverage rises quickly and saturates long before the run ends, while rooms are completed one by one.

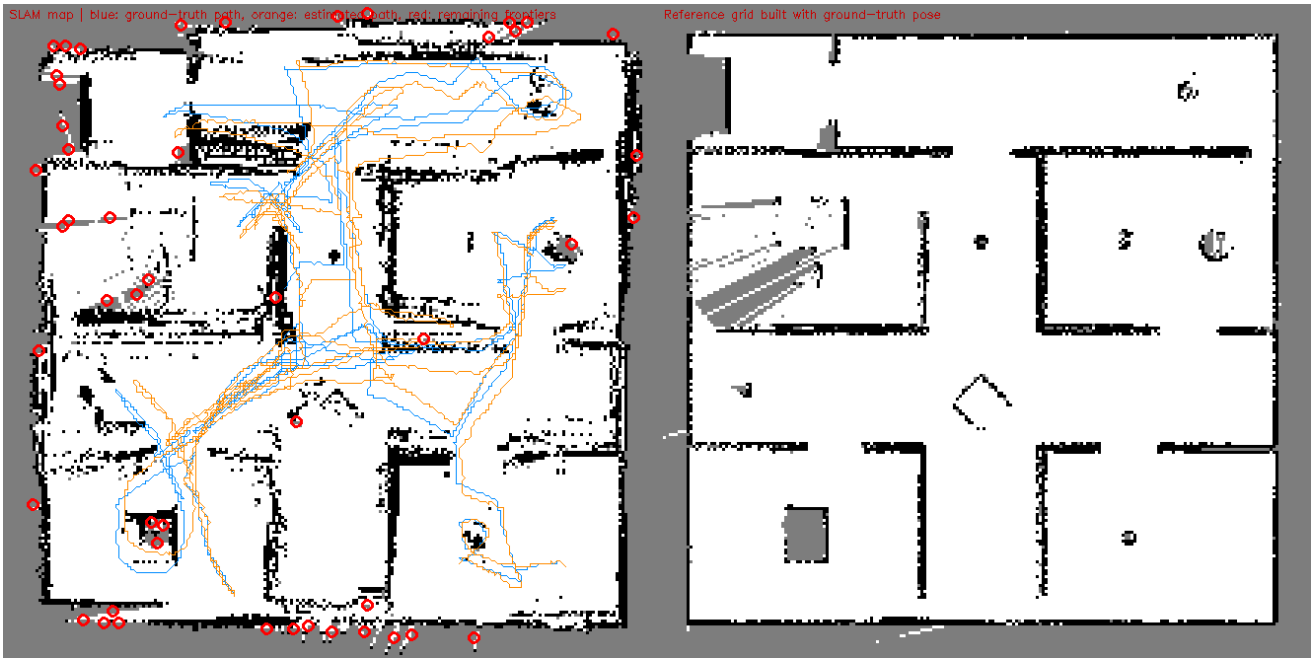


Figure 3. Final map (left) with the true path (blue), the estimated path (orange) and the remaining frontiers (red), and the reference map built with the true pose (right).

6.3 Localization

Table 9. Localization results against simulator ground truth.

Metric	Value	What it measures
ATE RMSE	31.0 cm	Typical position error
ATE median / maximum	27.1 / 63.1 cm	Median and worst error
ATE as share of path	0.20%	Error relative to distance
Final error	38.5 cm (0.24% of path)	Drift at the end
RPE translation	9.3 cm per metre (7.4%)	Local accuracy
RPE rotation	1.30 deg per metre	Local heading accuracy
Heading error mean / maximum	0.74 / 4.9 deg	Orientation error
Error first above 0.5 m / 1 m	255 s / never	Error never reached 1 m
Loop closures	68	Revisits used to correct drift
Scan-match acceptance	26.9% of 36000 updates	Diagnostic; rejection reasons are not recorded
Odometry distance vs true	134.7 m of 157.9 m	Diagnostic only (true path also contains motion the gait did not command)

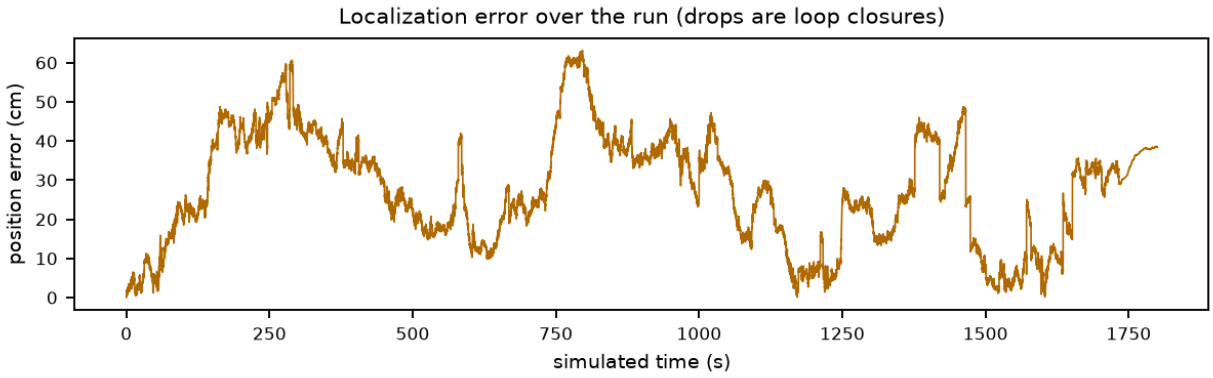


Figure 4. Localization error over the run. The error never exceeds about 60 cm; drops are loop closures.

Reading. There is no runaway drift: after 30 minutes the error is 0.24% of the path. Absolute error (31 cm) is larger than the 15 cm tolerance used for map scoring, and the local error per metre (7.4%) is high. The offline replay in Section 6.7 shows that odometry plus gyro alone has a lower local error but its heading drifts without bound, so scan matching buys global consistency at the cost of local jitter.

6.4 Mapping

Table 10. Mapping results against the real scene and against the true-pose map.

Metric	Against real scene	Against true-pose map
Accuracy	10.6 cm	12.9 cm
Completion	9.2 cm	6.4 cm
Precision / recall / F1 (15 cm)	73.9 / 81.8 / 77.7%	66.8 / 91.3 / 77.2%
Completion ratio (5 cm)	42.0%	54.1%
Free-space mapped outside the arena	0.59 m2	n/a
Map update time (median / p95)	20.3 / 36.9 ms	
Map memory	0.19 MB	

The same depth rays mapped with the *true* pose give 0.2 cm accuracy and 89.5% recall against the real scene. The sensor therefore allows an almost exact map; the difference to the 10.6 cm of the actual run comes from pose error, which smears and doubles walls (5407 occupied cells against 3184 in the true-pose map).

6.5 Object detection and knowledge graph

Table 11. Object detection against the scene's real objects.

Metric	Value
Real objects in the sensing band	13
Detections reported	113 (all labelled 'chair')
Detections on a real object	24 of 113 (21.2%)
Chair labels on the real chair	1 of 113 (the scene has 1 chair)
Knowledge graph	8 rooms, 17 doorway entries; every doorway recorded as connecting rooms 1 and 2; all room centres at (0, 0)

The detection result is expected from the method (Section 3.4): any obstacle blob of a suitable size receives the label 'chair', detections are never removed once reported, and no image information is used. This is a placeholder for the recognition stage of the full pipeline, not a recognition result.

6.6 Processing time and resources

Table 12. Time of one step call per agent (ms) and total.

Agent	Median	95th percentile	Maximum	Total (s)	Share of agent time
PerceptionAgent	9.14	17.43	1986	355	14%
SLAMAgent	10.62	23.64	85	440	17%
MappingAgent	20.30	36.93	179	805	31%
FrontierAgent	155.00	220.89	577	556	21%
PlanningAgent	0.42	127.62	1061	161	6%
LocomotionAgent	7.25	10.72	52	275	11%
SupervisorAgent	0.02	0.03	0	0	0%

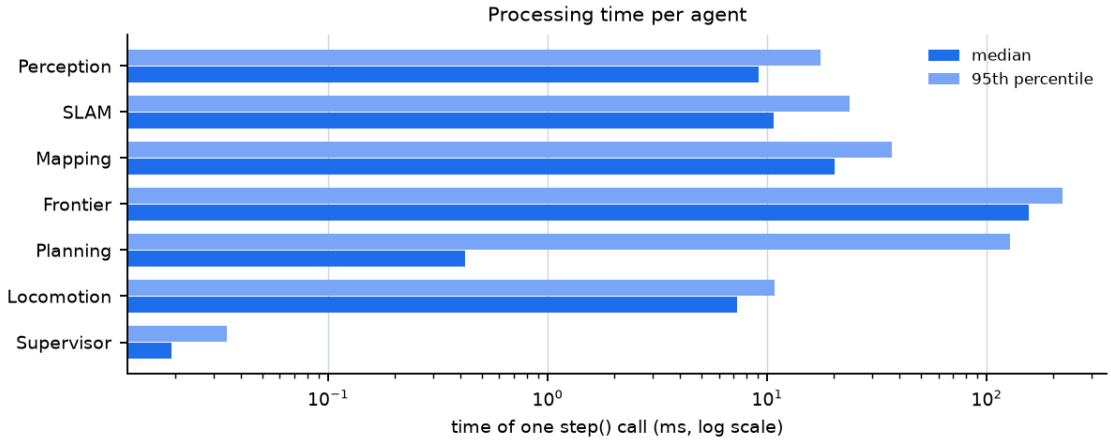


Figure 5. Median and 95th-percentile processing time per agent (logarithmic scale).

Table 13. Whole-system resources.

Metric	Value
Control-loop step (median / p95 / max)	91 / 380 / 2248 ms
Real-time factor	0.32 (1800 s simulated in 5666 s)
Share of wall time inside agents	46%
Process CPU (mean / p95)	86 / 106% of one core
Memory (mean / peak)	1037 / 2973 MB (includes the in-memory recording buffer)
GPU	not used

Every agent except frontier selection (median 155 ms, once per second) is well inside its time budget in the median. The weakness is the tail: Perception reached 1986 ms and Planning 1061 ms in single calls, giving control steps up to 2.2 s. The causes were not traced. Because the simulator waits for each step, these stalls cost wall time but do not change robot behaviour.

6.7 Component ablation

Table 14. Effect of removing one SLAM component, by offline replay of the same recording.

Configuration	ATE RMSE (cm)	Final error (cm)	RPE (%)	Heading error (deg)	Loop closures	Change in ATE
full system	31.0	38.5	7.4	0.74	68	-
without scan matching (odometry + gyro only)	264.8	135.7	1.8	44.62	0	+756%
without loop closure	54.4	90.1	6.2	0.71	0	+76%
without degeneracy filter	32.9	30.7	7.5	0.74	70	+6%
without Manhattan heading prior	168.9	215.9	7.5	18.78	71	+446%

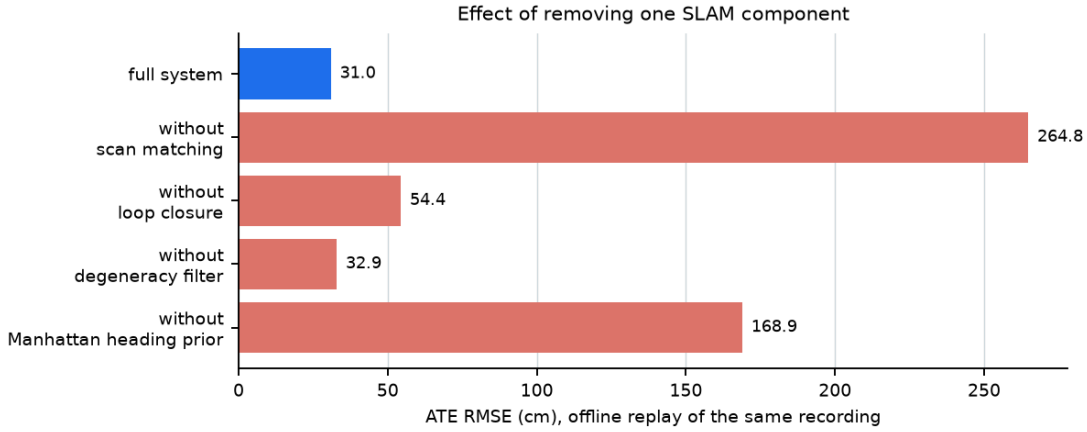


Figure 6. ATE with one component removed (same data, offline).

Scan matching, the Manhattan heading prior and loop closure each clearly help (+756%, +446% and +76% ATE when removed). The degeneracy filter shows no measurable benefit on this run (+6%, and a lower final error without it); it may matter in long corridors, which this arena has little of. The finding rests on one recording.

6.8 Classification of all metrics

Table 15. Classification of 38 metrics (10 Good, 12 Improve, 16 Poor).

Component	Metric	Value	Class
Exploration	Task completed	no	Poor
Exploration	Average speed	0.088 m/s	Poor
Exploration	Distance walked after coverage plateaued	46 % of path	Poor
Exploration	Room entries per room	2.25	Poor
Exploration	Stuck events	2.5 per 100 m	Improve
Exploration	Unreachable targets / paths planned	0.12 % of planner calls	Good
Localization	ATE as % of path length	0.20 %	Good
Localization	ATE RMSE	31.0 cm	Improve
Localization	Final drift	0.24 % of path	Good
Localization	Largest position error	63 cm	Improve

Component	Metric	Value	Class
Localization	Mean heading error	0.74 deg	Good
Localization	Relative error per metre (RPE)	7.4 % of 1 m	Poor
Localization	Scan-match acceptance (diagnostic)	26.9 %	Poor
Mapping	Wall accuracy	12.9 cm	Poor
Mapping	Wall completion	6.4 cm	Improve
Mapping	Completion ratio (5 cm)	54.1 %	Poor
Mapping	Precision	66.8 %	Poor
Mapping	Recall	91.3 %	Good
Mapping	F1	77.2 %	Improve
Mapping	Free-space IoU	81.2 %	Improve
Mapping	Map update time (p95)	36.9 ms	Good
Mapping	Map memory	0.19 MB	Good
Mapping and detection against the real scene	Map accuracy vs real walls/objects	10.6 cm	Poor
Mapping and detection against the real scene	Map recall vs real scene	81.8 %	Improve
Mapping and detection against the real scene	Map precision vs real scene	73.9 %	Improve
Mapping and detection against the real scene	Real floor mapped as free	88.2 %	Improve
Mapping and detection against the real scene	Free space mapped outside the arena	0.59 m2	Poor
Mapping and detection against the real scene	Object detection precision	21.2 %	Poor
Mapping and detection against the real scene	'Chair' labels on the real chair	1 of 113	Poor
Agents and real-time operation	SLAM update time (median)	10.6 ms	Good
Agents and real-time operation	Frontier selection (median)	155.0 ms	Improve
Agents and real-time operation	Planner time (p95)	127.6 ms	Improve
Agents and real-time operation	Slowest single agent call	1986 ms	Poor
Agents and real-time operation	Control-loop step (p95)	380 ms	Poor
Agents and real-time operation	Real-time factor	0.32 x	Poor
Agents and real-time operation	Process CPU (mean)	86 % of one core	Good
Agents and real-time operation	Memory (peak)	2973 MB	Improve
Agents and real-time operation	Telemetry overhead	0.004 % of loop step	Good

7. Comparison with published systems and the market

7.1 Method and comparability

Published values are quoted from the cited tables and product pages; where a value is the quotient of two published numbers it is marked derived. **No published system was evaluated under our conditions** (same robot, sensor, scene and protocol), so none is directly comparable. The closest are partial matches: the A1 SLAM paper (a quadruped with a 2D scanner, 5 m paths), the ANYmal D evaluation (a quadruped, 145 to 332 m, mostly outdoors) and the FARE exploration paper (wheeled robot, 3D LiDAR). The tables therefore show where the gap is, not a ranking. Differences in scene, sensing, ground truth and map type are listed in each table.

7.2 SLAM as a whole: pose accuracy, map accuracy and cost

Table 16. Published systems are RGB-D neural SLAM on the Replica dataset [26] with a GPU; maps are 3D surfaces. Ours is a 2D grid of 5 cm cells (which cannot resolve finer than about 2.5 cm), scored against the real scene.

System	Test conditions	Pose: ATE RMSE (cm)	Map: accuracy / completion (cm) / completion ratio (%)	Tracking per frame (ms)	Mapping per frame (s)	Map memory (MB)
Ours, run 1	simulated 8-room arena, 158 m; forward RGB-D to 2D scan + gait odometry + gyro; 2D occupancy grid scored against real scene walls and objects; CPU [this work]	31.0	10.6 / 9.2 / 42.0	10.6	0.020	0.19
MNE-SLAM	Replica scenes, RGB-D, neural 3D map, GPU [1]	0.398	1.681 / 1.509 / 97.0	22.1	0.14	24.45
Loopy-SLAM	Replica scenes, RGB-D, neural 3D map, GPU [1]	0.369	1.593 / 2.645 / 91.1	-	-	-
GO-SLAM	Replica scenes, RGB-D, neural 3D map, GPU [1]	0.404	2.091 / 2.415 / 93.2	-	-	-
ESLAM	Replica scenes, RGB-D, neural 3D map, GPU [1]	0.657	2.082 / 1.754 / 96.0	68.5	0.29	20.73
Co-SLAM	Replica scenes, RGB-D, neural 3D map, GPU [1]	1.059	2.104 / 2.082 / 93.4	54.8	0.15	26.39
Point-SLAM	Replica scenes, RGB-D, neural 3D map, GPU [1]	0.559	1.935 / 2.779 / 90.5	29.7	10.47	31.42

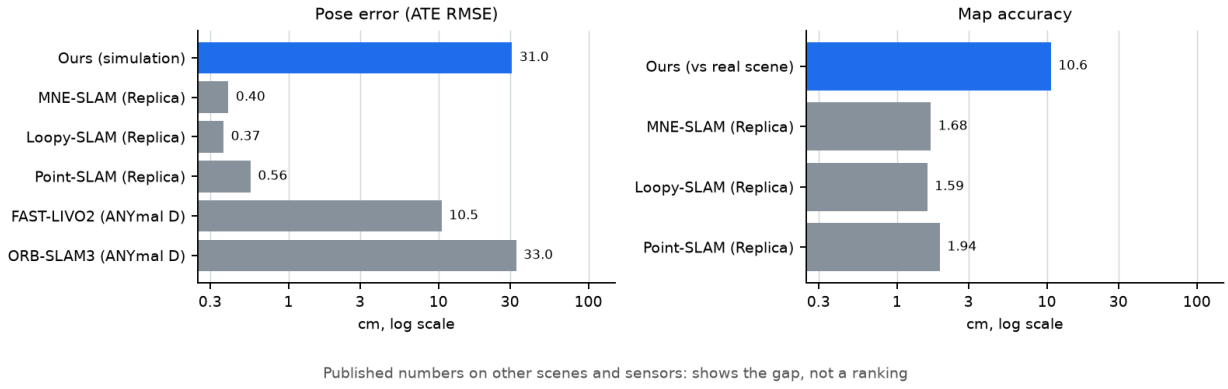


Figure 7. Pose error and map accuracy of our system beside published systems (log scale). Different scenes, sensors and map types: shows the gap, not a ranking.

7.3 Pose accuracy in multi-room scenes

Table 17. Ours: the robot's own arena. Published: Replica apartments 1, 2 and 0.

System	Test conditions	ATE RMSE (cm)
Ours, run 1	own 8-room arena, 158 m [this work]	31.0
MNE-SLAM	Replica multi-room apartments 1 / 2 / 0 [1]	1.02 / 0.59 / 0.48
ESLAM	Replica multi-room apartments 1 / 2 / 0 [1]	1.17 / 0.79 / 0.76
Loopy-SLAM	Replica multi-room apartments 1 / 2 / 0 [1]	1.43 / 0.60 / 0.64
ORB-SLAM3	Replica multi-room apartments 1 / 2 / 0 [1]	4.93 / 1.36 / 1.07
Swarm-SLAM	Replica multi-room apartments 1 / 2 / 0 [1]	5.56 / 5.61 / 1.80

System	Test conditions	ATE RMSE (cm)
CP-SLAM	Replica multi-room apartments 1 / 2 / 0 [1]	5.73 / 1.85 / 0.91

7.4 Pose accuracy on legged robots

Table 18. ORB-SLAM3 is described in [27]. ATE per path length for published systems is derived from the mean mission length (234.8 m).

System	Test conditions	Path (m)	ATE RMSE (cm)	ATE / path (%, derived)	CPU (%) / RAM (GB)
Ours, run 1	simulation, simulator pose [this work]	158	31.0	0.20	86 / 1.0
A1 SLAM	real Unitree A1, lab, motion capture [3]	5.0-5.3	4.3-14.5	0.8-2.9	-
Cartographer	real Unitree A1, lab, motion capture [3]	5.0-5.3	4.0-139.7	0.8-27.9	-
FAST-LIVO2, LiDAR + camera + IMU (Honeywell)	real ANYmal D, mostly outdoor, 7 missions [4]	145-332	10.5	0.04	125 / 3.91
ORB-SLAM3, stereo (global shutter)	real ANYmal D, mostly outdoor, 7 missions [4]	145-332	33.0	0.14	290 / 1.16
RTAB-Map, stereo (global shutter)	real ANYmal D, mostly outdoor, 7 missions [4]	145-332	57.9	0.25	116 / 0.75
RTAB-Map, RGB-D (ZED2i)	real ANYmal D, mostly outdoor, 7 missions [4]	145-332	241.3 (successful runs only)	1.03	-
ORB-SLAM3, RGB-D (ZED2i)	real ANYmal D, mostly outdoor, 7 missions [4]	145-332	821.4	3.50	-

7.5 Processing time per update

Table 19. Different tasks and hardware: 2D scan matching against feature-based or neural tracking. This says nothing about accuracy.

System	Quantity	Value	Hardware / note
Ours, run 1	SLAM update, median	10.6 ms	CPU only, AMD Ryzen 7 7735HS with Radeon Graphics [this work]
ORB-SLAM2 (RGB-D)	tracking per frame	25.58 ms	Intel Core i7-4790, TUM fr3_office [2]
MNE-SLAM	tracking per frame	22.1 ms	authors' hardware [1]
A1 SLAM / Cartographer	optimization per scan	1.65 / 0.96 ms	optimizer call only [3]

7.6 Exploration speed

Table 20. Distances and times are not comparable (different environments and robots, and our run did not complete). Speed is a property of the robot.

System	Test conditions	Distance (m)	Time (s)	Average speed (m/s)
Ours, run 1	quadruped, forward RGB-D, 10 x 10 m arena [this work]	158	1800 (not completed)	0.088
DSVP	wheeled robot, 16-channel 3D LiDAR, Gazebo indoor, max 2 m/s [5]	1511	931	1.62 (derived)
TARE	wheeled robot, 16-channel 3D LiDAR, Gazebo indoor, max 2 m/s [5]	1209	658	1.84 (derived)

System	Test conditions	Distance (m)	Time (s)	Average speed (m/s)
ARiADNE	wheeled robot, 16-channel 3D LiDAR, Gazebo indoor, max 2 m/s [5]	1053	610	1.73 (derived)
HEADER	wheeled robot, 16-channel 3D LiDAR, Gazebo indoor, max 2 m/s [5]	1030	576	1.79 (derived)
FARE (LLM + RL)	wheeled robot, 16-channel 3D LiDAR, Gazebo indoor, max 2 m/s [5]	1048	590	1.78 (derived)
ANYmal (product page)	quadruped, 360 deg LiDAR + depth cameras [7]	-	-	0.75 normal walking
Spot (product page)	quadruped, 360 deg terrain sensing [6]	-	-	1.6 maximum
Go2 (product page)	quadruped, 4D LiDAR [8]	-	-	about 5 maximum (lab)

7.7 Commercial platforms

Table 21. Capabilities from manufacturer pages. None of this is measured by us; endurance, weatherproofing and price do not apply to a simulation.

System	Platform	Sensing	Explores unknown space	Needs prior map or path	Speed	Endurance	Compute	Protection
Ours, run 1	quadruped (simulation)	forward RGB-D camera, gait odometry, gyro	Yes, frontier-based (did not complete in our test)	No	0.088 m/s measured	not measured (simulation)	CPU laptop, 86 % of one core	-
Boston Dynamics Spot [6]	quadruped	360 deg terrain sensing (4 m range); payload ports	No (needs a recorded map)	Yes (recorded map / route)	1.6 m/s max	~90 min (564 Wh)	not stated	IP54
ANYbotics ANYmal [7]	quadruped	360 deg LiDAR, 6 depth cameras, 2 teleop cameras; thermal/zoom payload	Not stated on product page	Not stated	0.75 m/s normal	90-120 min (2 km)	2x Intel Core i7	IP67
Unitree Go2 (AIR/PRO/X) [8]	quadruped	4D LiDAR L2 (360 x 96 deg); depth camera on EDU only	No (user specifies the path)	Yes (user-specified path)	~5 m/s max (lab)	1-2 h (2-4 h EDU)	8-core CPU (EDU: optional Orin)	not stated
Unitree B2 [9]	quadruped	3D LiDAR x1, depth cameras x2, optical cameras x2 (varies by configuration)	Not stated	Not stated	> 6 m/s (special configuration; limited in practice)	4-6 h	Intel Core i5 / i7; optional Orin NX	IP67
DEEP Robotics X30 [10]	quadruped	integrated perception (sensors not listed)	Not stated	Not stated	>= 4 m/s	2.5-4 h	not stated	IP67
Emesent Hovermap (payload, mainly drones) [11]	sensor payload	LiDAR	Yes (autonomous exploration mode)	Not stated	not stated	not stated	onboard (not specified)	not stated

Spot and Go2 are documented as following a recorded map or a user-specified path. The ANYmal, B2 and X30 pages do not state whether the robot explores unknown space. The Emesent Hovermap payload (mainly used on drones) does offer an autonomous exploration mode, so autonomous exploration is not unique to this system. Every commercial robot is far faster than ours.

7.8 A like-for-like reference for later work

Table 22. TUM RGB-D Pioneer sequences (a wheeled robot with a forward Kinect on a flat floor, motion-capture truth): published ATE RMSE in metres. Our system has not been run on these.

System	pioneer_360	pioneer_slam	pioneer_slam2	pioneer_slam3	Average
Ours	not yet run	not yet run	not yet run	not yet run	not yet run
ORB-SLAM3 (RGB-D) [12]	1.059	2.118	1.709	1.992	1.720
OpenVSLAM (RGB-D) [12]	0.048	0.188	1.224	0.106	0.392
RTABMap (RGB-D) [12]	0.038	0.351	0.044	0.716	0.287

This is the public dataset closest to our robot. Running our pipeline on it would give the first number measured under the same conditions as published systems. Hand-held and drone datasets (TUM hand-held, Replica, EuRoC) involve full 3D motion and do not fit a planar 2D SLAM.

7.9 Where the system stands

Table 23. Position relative to published systems.

Dimension	Standing	Evidence
Cost (memory, update time)	Ahead	0.19 MB against 20 to 31 MB; 10.6 ms against 22 to 69 ms (different tasks)
Drift per path length	Same order	0.20% against derived 0.14% for ORB-SLAM3 stereo and 0.04% for FAST-LIVO2 on a real ANYmal D
Absolute pose and map accuracy	Behind by about an order of magnitude or more	ATE 31.0 cm against 0.4 to 1.1 cm; map accuracy 10.6 cm against 1.6 to 2.1 cm (Replica RGB-D systems)
Exploration speed	Behind	0.088 m/s against 1.6 to 1.8 m/s (FARE planners, derived) and 0.75 to over 6 m/s (commercial robots)
Mission completion	Not achieved	Published exploration systems report completion time; ours did not complete
Object recognition	Not comparable yet	Detection is a size rule; no recognition model is integrated

8. Analysis and discussion

8.1 Why the mission did not complete

The interior was effectively mapped early: coverage reached 97% of its final value at 930 s. The remaining 46% of the path (73.2 m) added 2.1 points. The cause is in the frontier logic combined with the map:

- A frontier is any free cell next to unknown space. Of the 48 frontiers left at the end, **21 lie outside the outer wall** and 13 within 0.5 m of it. The map has 0.59 m2 of free space beyond the wall: depth rays leak past walls at grazing angles. Even the reference map built with the true pose has 34 frontiers (9 outside), so the leak comes from the sensor model, and pose error makes it worse.
- A frontier within 0.3 m of reachable floor counts as reachable. The robot drives to the wall; looking from inside cannot clear a frontier beyond the wall, so the target is blacklisted after a timeout.
- The blacklist is cleared when it exceeds 40 entries or no candidate remains, so the same targets return (single targets were planned 11 to 21 times; 1,814 paths were planned in 1,800 s).
- Room labels change as the map grows (18 room-entered events for 8 rooms), which restarts room sweeps.

These are diagnoses from the map, the code and the log; they are not yet confirmed by a run with the fixes applied.

8.2 What works

- No runaway drift (0.24% of the path at the end, never above 63 cm) with a gyro that drifts and an odometry that is only calibrated once.
- The heading prior and scan matching are decisive; removing either destroys the estimate.
- The system is cheap: 86% of one CPU core and a 0.19 MB map, without a GPU.
- Walls are found reliably (recall 82%), and 88% of the real floor was mapped.

8.3 Weaknesses

- The mission never completes (false frontiers), and speed is low.
- Local pose error is high (7.4% per metre), and walls are smeared (10.6 cm).
- The object detector is not a recognizer, and the knowledge graph contains placeholders for doorway connectivity and room centres.
- The control loop is not real time (0.32x) and stalls for up to 2.2 s.

8.4 Agent execution model

The split into agents is a clean software structure but not a performance mechanism. The agents run sequentially in one thread and use 46% of wall time; the rest is outside them (simulator stepping, image transfer, display, and a map query in the main loop on every step), and where exactly it goes has not been profiled. Because the simulator waits for the program each step, speeding up the agents changes wall time only. Running agents in parallel would require processes or a framework such as ROS 2, and would change timing behaviour, so a new baseline would be needed.

8.5 Threats to validity

- **One run in one arena.** No variability or generalization can be claimed.
- **Simulation.** Sensor noise, slip and lighting are simulated; the gait speed is not that of a real robot.
- **Start at the true pose.** SLAM begins at the true start pose, so no global alignment error is included.
- **Calibration against ground truth.** The gait-speed curve was fitted offline from an early recording that used ground truth.
- **Turning shortcut.** The body orientation is set directly when turning.
- **Reference maps.** The first reference is built from the same sensor; the second (real scene) is independent but rasterized at 5 cm and its object orientations are inferred.
- **Classification limits.** Several limits are engineering targets proposed here.
- **Comparisons.** Published numbers come from other scenes, sensors and protocols.

9. Roadmap

Nothing below has been applied yet; the baseline was taken first so that each change can be judged against it. The full register with evidence and acceptance checks is in docs/project/FIXES.md.

Table 24. Planned work in priority order.

Phase	Work	Fix IDs	Success check
1. Make exploration finish	Ignore frontiers outside walls or without nearby floor; keep the blacklist; freeze room labels; end a run on completion or lack of progress	F1 to F4, F30	Mission reports completion; no frontiers outside the wall
2. Measure what is still unknown	Profile where wall time goes; run uncapped for time to completion; repeat runs with different seeds; test a second arena	F12, F18, F20	Completion time; spread over runs
3. Accuracy	Reduce local jitter of scan matching; reduce wall smearing; tighten absolute error	F7 to F9	RPE below 3%; wall accuracy at most 10 cm; ATE at most 15 cm

Phase	Work	Fix IDs	Success check
4. Speed and real time	Raise gait speed; remove stalls; speed up frontier and mapping steps	F5, F13 to F15	Real-time factor at least 1; control step p95 at most 100 ms
5. Perception	Replace the size rule with recognition from the colour image; fix knowledge-graph placeholders	F28, F29	Detection precision at least 80%
6. External validation	Run on the TUM Pioneer sequences; run Cartographer on the recording; same-arena baseline explorer; hardware	F25 to F27, F24	Measured numbers beside published ones
7. Full pipeline	Anomaly detection, event triggering, tamper-evident logging, synchronization, battery-aware return (later project stages)	outside this repository	Per the project plan

10. Conclusion

A seven-agent, CPU-only exploration and SLAM system was built and measured honestly. The baseline shows a localization estimate that does not drift (0.20% of the path), a cheap implementation and clear evidence that scan matching, the heading prior and loop closure each earn their place. It also shows what must be fixed before the system can be called complete: it does not finish exploring because of false frontiers beyond the walls, it is slow, its absolute accuracy is about an order of magnitude behind published RGB-D systems, and its object detection is a placeholder. Because the baseline, the measurement framework and the fix register are in place, each improvement can now be tested against the same measurements.

References

[1] T. Deng, G. Shen, C. Xun, S. Yuan, T. Jin, H. Shen, Y. Wang, J. Wang, H. Wang, D. Wang, W. Chen, "MNE-SLAM: Multi-Agent Neural SLAM for Mobile Robots", CVPR 2025, pp. 1485-1494. https://doi.org/10.1109/CVPR52734.2025.00146
[2] R. Mur-Artal, J. D. Tardos, "ORB-SLAM2: an Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras". https://arxiv.org/abs/1610.06475
[3] J. Chen, F. Dellaert, "A1 SLAM: Quadruped SLAM using the A1's Onboard Sensors". https://arxiv.org/abs/2211.14432
[4] R. Corlito, F. Schmidt, N. Seibert, et al., "Sensor Configuration Matters: A Systematic Evaluation of Multimodal SLAM on Quadruped Robots" (ANYmal D, GrandTour dataset). https://arxiv.org/abs/2606.19067
[5] S. Liao, X. Lv, J. Lew, S. Zhang, et al., "FARE: Fast-Slow Agentic Robotic Exploration". https://arxiv.org/abs/2601.14681
[6] Boston Dynamics, Spot product specifications and Spot SDK 5.2.0 autonomy documentation (GraphNav, Directed Exploration). https://bostondynamics.com/products/spot/
[7] ANYbotics, ANYmal product page and technical specifications. https://www.anybotics.com/robotics/anymal/
[8] Unitree Robotics, Go2 product page and specification table. https://www.unitree.com/go2
[9] Unitree Robotics, B2 product page and specification table. https://www.unitree.com/b2
[10] DEEP Robotics, X30 product page and specification table. https://en.deeprobotics.cn/en/index/product3.html
[11] Emesent, Hovermap autonomy page. https://www.emesent.com/autonomy
[12] A. Merzlyakov, S. Macenski, "A Comparison of Modern General-Purpose Visual SLAM Approaches". https://arxiv.org/abs/2107.07589
[13] J. Sturm, N. Engelhard, F. Endres, W. Burgard, D. Cremers, "A Benchmark for the Evaluation of RGB-D SLAM Systems", Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS), 2012. https://cvg.cit.tum.de/data/datasets/rgbd-dataset
[14] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, J. J. Leonard, "Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age", IEEE Trans. Robotics, vol. 32, no. 6, pp. 1309-1332, 2016.
[15] E. Rohmer, S. P. N. Singh, M. Freese, "V-REP: A Versatile and Scalable Robot Simulation Framework", Proc. IEEE/RSJ IROS, pp. 1321-1326, 2013 (the simulator now named CoppeliaSim).
[16] P. J. Besl, N. D. McKay, "A Method for Registration of 3-D Shapes", IEEE Trans. Pattern Analysis and Machine Intelligence, vol. 14, no. 2, pp. 239-256, 1992.
[17] A. Censi, "An ICP Variant Using a Point-to-Line Metric", Proc. IEEE Int. Conf. Robotics and Automation (ICRA), pp. 19-25, 2008.
[18] G. Grisetti, R. Kuemmerle, C. Stachniss, W. Burgard, "A Tutorial on Graph-Based SLAM", IEEE Intelligent Transportation Systems Magazine, vol. 2, no. 4, pp. 31-43, 2010.
[19] W. Hess, D. Kohler, H. Rapp, D. Andor, "Real-Time Loop Closure in 2D LIDAR SLAM", Proc. IEEE ICRA, pp. 1271-1278, 2016.
[20] J. M. Coughlan, A. L. Yuille, "Manhattan World: Compass Direction from a Single Image by Bayesian Inference", Proc. IEEE Int. Conf. Computer Vision (ICCV), 1999.
[21] H. Moravec, A. Elfes, "High Resolution Maps from Wide Angle Sonar", Proc. IEEE ICRA, pp. 116-121, 1985.
[22] A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation", Computer, vol. 22, no. 6, pp. 46-57, 1989.
[23] B. Yamauchi, "A Frontier-Based Approach for Autonomous Exploration", Proc. IEEE Int. Symp. Computational Intelligence in Robotics and Automation (CIRA), pp. 146-151, 1997.
[24] P. E. Hart, N. J. Nilsson, B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths", IEEE Trans. Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968.
[25] R. Kuemmerle, B. Steder, C. Dornhege, M. Ruhnke, G. Grisetti, C. Stachniss, A. Kleiner, "On Measuring the Accuracy of SLAM Algorithms", Autonomous Robots, vol. 27, no. 4, pp. 387-407, 2009.
[26] J. Straub et al., "The Replica Dataset: A Digital Replica of Indoor Spaces", arXiv:1906.05797, 2019.
[27] C. Campos, R. Elvira, J. J. G. Rodriguez, J. M. M. Montiel, J. D. Tardos, "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM", IEEE Trans. Robotics, vol. 37, no. 6, pp. 1874-1890, 2021.

Appendix A. Repository structure and reproduction

Table 25. Repository layout.

Path	Contents
main_agentic.py	Entry point: starts the simulation, runs the agents, records the run
agents/	The seven agents, message bus, knowledge graph, episodic memory
core/	Algorithms and simulator interface: slam, grid, planner, robot, sensors, viz
evaluation/	Telemetry, report generation, scene scoring, ablation, replay, audit, comparison tables and published numbers, PDF and figure builders
scene/	The CoppeliaSim scene
tests/	Formula and algorithm tests
runs/	One folder per measured run
docs/	results (PDFs), project (objectives, baseline, fixes), course (panel documents), images (figures)

To reproduce: install the requirements, open scene/robot_working.ttt in CoppeliaSim (do not press play), and run *python main_agentic.py*; then *python -m evaluation.ablate runs/<run_id>* and *python -m evaluation.report runs/<run_id>*. The PDFs are rebuilt with *python -m evaluation.results_pdf*, *baseline_pdf* and *project_report*.

Appendix B. Key parameters

Table 26. Parameters of the main algorithms.

Component	Parameter	Value
Camera	Field of view / range	60 degrees / 0.1 to 8 m
Perception	Self-mask radius; map height band; obstacle minimum height; point stride	0.22 m; -0.08 to 0.60 m; 0.04 m; 4
Scan matching	Residual limit; step limit; odometry agreement gate	5 cm; 0.8 m and 120 degrees; 1 cm plus the step length
Heading	ICP yaw weight; yaw agreement; Manhattan gain and limit	0.1; 3 degrees; 0.1 and 15 degrees
Loop closure	Keyframe spacing; search radius; minimum age; residual; overlap; maximum correction	0.5 m or 25 degrees; 2.5 m; 12 s; 3 cm; 60%; 0.3 m
Grid	Size; resolution; hit / miss update; limits; thresholds	220 x 220; 5 cm; +0.85 / -0.4; +/-2; occupied 1.0, free -0.5
Frontier	Information gain; distance weight; hemisphere; target hold	cluster size (max 20) + 0.3 per unknown cell in 9 x 9; 3.8 per metre; +/-95 degrees; 9 s
Planner	Hard clearance; connectivity	15 cm; 8-connected with soft wall penalty
Locomotion	Speed commands; stuck check; progress timeout	1.0, 0.75, 0.35; 5 cm per second (two failures); 15 s
Gyro	Noise; bias	0.10 deg/s; 0.05 deg/s

Appendix C. Findings register

All 31 findings (F1 to F31) with evidence, planned change and acceptance check are maintained in docs/project/FIXES.md. Fix F31 (end-of-run report failure) has been applied; all others are open.